

7. Machine Learning for Price Forecasting

Why Move Beyond Linear Models?

In Chapter 6, we built LEAR — a linear model that predicts electricity prices as a weighted sum of features. LEAR is a strong baseline, and in many forecasting competitions it performs surprisingly well. But it has a fundamental limitation baked into its very structure: it can only model price as a **linear function** of the inputs. If price doubles, LEAR assumes the feature driving the change also doubled — and vice versa.

Real electricity prices do not behave this way.

Linear model: A model that predicts the output as a weighted sum of its inputs: $y^* = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$. The relationship between any input and the output is a straight line (or a flat plane in higher dimensions). Linear models cannot capture curves, thresholds, or interactions between features unless those are explicitly engineered into the input.

Recall the **hockey-stick relationship** from Chapter 3: when net load (demand minus renewables) is below a certain threshold, the price is relatively flat — cheap coal and gas baseload generators set the marginal price. But when net load crosses the threshold, expensive peaking generators are dispatched and the price shoots upward exponentially. This is an inherently **nonlinear** relationship — a curve with a sharp bend, not a straight line.

A linear model must approximate this curve with a single straight line. It will inevitably undershoot during spikes (the line cannot bend sharply upward) and overshoot during quiet periods (the line is pulled upward by the spike observations). No amount of feature engineering can fix this — the limitation is structural.

Nonlinear model: A model that can learn curved, bent, or discontinuous relationships between inputs and outputs. Unlike a linear model, it does not assume that doubling an input doubles the output. Decision trees, gradient-boosted trees, and neural networks are all nonlinear models.

The core argument for machine learning: Electricity price formation is governed by the merit-order dispatch process — a step function that is flat in some regions and vertical in others. A model that can learn step functions, thresholds, and sharp bends will systematically outperform one limited to straight lines. Tree-based models and neural networks can learn these shapes directly from the data, without requiring the analyst to specify the functional form in advance.

This chapter covers two families of nonlinear models: **gradient-boosted trees (GBT)** — the workhorse of modern tabular data modelling — and **neural networks** — powerful but often unnecessarily complex for this problem. We then extend both to **quantile regression**, which produces probabilistic forecasts rather than single point predictions.

Decision Trees: The Building Block

Before we can understand gradient boosting, we need to understand its fundamental building block: the **decision tree**.

Decision tree: A model that makes predictions by asking a sequence of yes/no questions about the input features. Each question splits the data into two groups, and the process repeats until the groups are small enough to make a prediction. The final prediction for each group is typically the average of all observations that land in that group. The name comes from the tree-like branching structure of these sequential decisions.

Think of a decision tree like a game of twenty questions. The tree asks: "Is net load above 8,000 MW?" If yes, it asks: "Is the hour after 5 PM?" If yes again, it predicts a high price. If no, it predicts a moderate price. Each question divides the data into more specific groups, and each group gets its own prediction.

How a Decision Tree Learns

A decision tree is built (or "grown") by a simple recursive algorithm:

Start with all the data. Consider every possible feature and every possible split point for that feature. For example: "net load > 7,500 MW" or "hour > 14" or "temperature > 35°C."

Choose the best split. The "best" split is the one that produces the biggest reduction in prediction error. Formally, we minimise the **sum of squared errors** within each of the two groups created by the split. The split that creates the most internally homogeneous groups wins.

Repeat recursively. Apply the same process to each of the two groups, creating further subdivisions. Each subdivision creates more specific regions with more accurate predictions.

Stop when a criterion is met. Common stopping criteria include: maximum tree depth (how many levels of questions), minimum number of observations in a leaf (final group), or when further splitting does not improve predictions enough to justify the added complexity.

Predict the mean. Once the tree is fully grown, each leaf (terminal node) predicts the mean of the target variable for all training observations that land in that leaf.

Leaf (terminal node): The final node of a decision tree branch, where a prediction is made. Every input observation eventually reaches exactly one leaf by following the yes/no questions down the tree. The leaf's prediction is the average target value of all training observations that reached that

leaf.

Split point: A threshold value for a feature that divides data into two groups. For example, "net load > 8,000 MW" is a split point that sends observations with net load above 8,000 to one branch and observations at or below 8,000 to another. The algorithm searches over all possible features and all possible threshold values to find the optimal split at each step.

Why Trees Are Natural for Price Forecasting

Decision trees have properties that make them particularly well-suited to electricity price forecasting:

Nonlinearity by construction. The hockey-stick relationship is literally a tree with one split on net load — prices are flat below the threshold and high above it. No feature engineering needed.

Automatic interaction detection. "If net load is high AND the hour is 6 PM AND temperature exceeds 35°C, predict a very high price" emerges naturally from nested splits. A linear model would need you to manually create the feature "net_load × hour × temperature."

Robustness to outliers. Because trees make predictions by averaging within regions, a single extreme observation in one leaf does not affect predictions in other leaves. By contrast, a single extreme price can dramatically shift the coefficients of a linear model.

No scaling needed. Trees only care about the *rank order* of feature values, not their magnitude. Whether temperature is in Celsius or Fahrenheit, the same splits are chosen. Linear models are sensitive to feature scaling.

The Problem with Single Trees

Despite their appealing properties, single decision trees have a critical weakness: **high variance**. Small changes in the training data produce very different trees. Add a few observations, remove a few others, and the tree might choose entirely different features and split points — producing wildly different predictions.

Variance (in model terms): The sensitivity of a model's predictions to changes in the training data. A high-variance model gives very different predictions when trained on slightly different datasets. A low-variance model gives similar predictions regardless of which specific observations are in the training set. Variance is one side of the **bias-variance tradeoff**: reducing variance typically increases bias (the model's inability to capture the true relationship), and vice versa.

Bias-variance tradeoff: A fundamental concept in machine learning. **Bias** is the error from simplifying assumptions — a linear model has high bias when the true relationship is nonlinear. **Variance** is the error from sensitivity to training data — a deep decision tree has high variance because it fits noise. The total error is approximately $\text{bias}^2 + \text{variance}$. The goal is to find the model complexity that minimises the total, not either component alone.

This is where **ensemble methods** come in — techniques that combine many trees to reduce variance while preserving the low bias of nonlinear models.

Gradient Boosting: From Weak to Strong

The Ensemble Idea

Ensemble method: A technique that combines multiple "weak" models into a single "strong" model. The idea is that individual models make different errors, and averaging or combining their predictions cancels out many of these errors. The two main ensemble strategies are **bagging** (training many models in parallel on random subsets of the data, then averaging — e.g., Random Forest) and **boosting** (training models sequentially, each correcting the errors of its predecessor).

Gradient boosting is the most powerful ensemble method for tabular data. The name reflects two ideas: "gradient" refers to the use of gradient descent to minimise the loss function, and "boosting" refers to the sequential improvement of predictions.

How Gradient Boosting Works

The core idea is simple: instead of building one big tree, build many small trees, each one correcting the mistakes of all the trees before it. Here is the process, step by step:

Start with a simple prediction. Initialise the model with a constant prediction — typically the mean of the training target. This is the "zeroth" model: $y^{*0} = \text{mean}(y)$.

Compute the residuals. Calculate the difference between the actual values and the current predictions: $r^1_i = y_i - y^{*0}$. These residuals represent what the current model gets wrong.

Fit a small tree to the residuals. Train a shallow decision tree (typically 3–8 levels deep) to predict the residuals, not the original target. This tree learns the *patterns in the errors* — where and how the current model is wrong.

Update the predictions. Add the new tree's predictions to the current model, scaled by a small number called the **learning rate**: $y^{*1} = y^{*0} + \eta \cdot f^1(x)$, where η is typically 0.01 to 0.1.

Repeat. Compute new residuals $r_i^2 = y_i - y_i^{*1}$, fit another tree, update the predictions. Each iteration reduces the remaining error.

Continue for B rounds. After B trees, the final prediction is the sum of all trees' contributions:

$$y^* = y^{*0} + \eta \cdot f^1(x) + \eta \cdot f^2(x) + \dots + \eta \cdot f^B(x) = y^{*0} + \eta \cdot \sum_{b=1}^B f^b(x)$$

Learning rate (η): A small positive number (typically 0.01–0.1) that controls how much each new tree contributes to the ensemble. A smaller learning rate means each tree makes a smaller correction, requiring more trees to reach the same level of accuracy — but the final model generalises better because it takes smaller, more careful steps. Think of it like learning a skill: taking many small practice steps produces more reliable mastery than a few large leaps.

Residual: The difference between the actual value and the model's current prediction: $r_i = y_i - y_i^*$. In gradient boosting, each successive tree is trained to predict the residuals of the current ensemble — it learns to correct the errors that the existing trees have not yet captured.

Intuitive example — correcting an essay: Imagine you write a first draft of an essay. A colleague reads it and marks the errors (residuals). You fix only those errors, producing draft 2. A different colleague reads draft 2 and marks its remaining errors (smaller now). You fix those, producing draft 3. After ten rounds of editing, the essay is much better than any single editor could have made it — each editor focused on what was still wrong after all previous fixes. Gradient boosting works the same way: each tree is an "editor" that focuses on what the previous trees got wrong. The final prediction is the original draft plus all the corrections.

Why boosting beats single trees: Each individual tree in a boosted ensemble is deliberately kept small and weak — it can only learn simple patterns. But the sequential error-correction means the ensemble as a whole can learn arbitrarily complex patterns. The variance problem of single trees is solved by the small tree size (each tree is too simple to overfit much), while the bias problem is solved by having many trees that collectively capture the full complexity of the data.

The "Gradient" in Gradient Boosting

The word "gradient" appears because the algorithm is actually performing **gradient descent** in function space. For squared error, the negative gradient of the loss function with respect to the prediction is simply the residual — which is why fitting to residuals works. But gradient boosting generalises to any differentiable loss function: instead of fitting to residuals, each tree fits to the negative gradient of the chosen loss function. This is what enables quantile regression (discussed later), where the loss function is the asymmetric pinball loss rather than squared error.

Gradient descent: An optimisation algorithm that iteratively adjusts parameters in the direction that reduces a loss function most steeply. Imagine standing on a hilly landscape in fog — you cannot see the bottom of the valley, but you can feel the slope under your feet. Gradient descent says: take a step downhill (in the steepest direction), reassess, repeat. Each step gets you closer to the lowest point. The "gradient" is the mathematical description of the slope.

LightGBM: The Industry Standard

Why LightGBM?

Several software implementations of gradient boosting exist (XGBoost, CatBoost, LightGBM, scikit-learn's GradientBoostingRegressor), but **LightGBM** (Light Gradient Boosting Machine) has become the dominant choice for tabular data modelling, including electricity price forecasting. Developed by Microsoft Research, it is faster, more memory-efficient, and often more accurate than its predecessors.

LightGBM: A highly optimised gradient boosting framework developed by Microsoft. It builds decision tree ensembles using innovations (histogram-based splitting, leaf-wise growth, native categorical support) that make it significantly faster than earlier implementations while maintaining or improving accuracy. It is the standard tool for tabular machine learning in industry and forecasting competitions.

Key Innovations

LightGBM's speed and accuracy come from three main innovations over earlier implementations like XGBoost:

Histogram-based splitting. When deciding where to split a feature, a naive implementation would test every unique value of the feature — which could be millions of values for continuous features like price or temperature. LightGBM instead **bins** continuous features into a fixed number of discrete buckets (typically 255), reducing the number of candidate splits from millions to hundreds. The accuracy loss from binning is negligible, but the speed improvement is dramatic — often 10× or more.

Histogram binning: The process of dividing a continuous variable into a fixed number of discrete buckets (bins). For example, if temperature ranges from 0°C to 45°C and we use 255 bins, each bin covers approximately 0.18°C. The tree only considers splitting at bin boundaries, not at every unique temperature. This dramatically reduces computation with minimal loss of information.

Leaf-wise growth. Traditional gradient boosting grows trees **level-by-level** (breadth-first): all nodes at depth 1 are split before any node at depth 2. This produces balanced trees where every branch has the same depth. LightGBM instead grows **leaf-wise** (best-first): at each step, it splits the leaf with the highest potential loss reduction, regardless of its depth. This produces unbalanced trees — some branches go deep where the data is complex, while others stop shallow where the data is simple. Leaf-wise growth typically produces more accurate trees with

fewer leaves.

Native categorical feature support. Many features in price forecasting are **categorical** — hour of day (0–23), day of week (1–7), season (1–4). Traditional approaches convert these to **one-hot encoding** (a separate binary column for each category), which is wasteful and fragments the data. LightGBM handles categorical features directly, finding the optimal partition of categories into two groups at each split. This is more statistically efficient and produces better splits.

One-hot encoding: A representation of a categorical variable using binary (0/1) columns, one per category. "Hour = 14" becomes a vector of 24 numbers, all zero except position 14. This is the standard way to feed categorical data into linear models and some tree implementations. LightGBM avoids this by handling categories natively.

Hyperparameters That Matter

A **hyperparameter** is a setting that you choose before training, rather than something the model learns from the data. LightGBM has many hyperparameters, but only a handful have a material effect on performance for price forecasting.

Hyperparameter: A configuration value set by the analyst before training begins. Unlike model parameters (e.g., tree split points, which are learned from data), hyperparameters control the learning process itself — how many trees to build, how large each tree should be, how fast to learn. Choosing good hyperparameters is called **hyperparameter tuning** and is typically done by trying many combinations and selecting the one that performs best on a validation set.

Parameter	Typical range	What it controls	Effect of increasing
n_estimators	100–2,000	Number of trees in the ensemble	More capacity to learn complex patterns; slower training; risk of overfitting if too large
learning_rate	0.01–0.1	Step size for each tree's contribution	Faster convergence but worse generalisation; fewer trees needed
max_depth	3–8	Maximum depth of each tree	More capacity for interactions and nonlinearity; greater overfitting risk
num_leaves	15–127	Maximum leaves per tree (overrides depth)	Direct control of tree complexity; 2 ^(max_depth) is the upper bound
min_child_samples	5–50	Minimum observations required in each leaf	More regularisation; prevents the tree from creating tiny, overfitting leaves
subsample	0.5–1.0	Fraction of training data used for each tree	Lower values add randomness, reducing overfitting (like bagging)
colsample_bytree	0.5–1.0	Fraction of features considered for each tree	Lower values force diversity among trees; prevents any one feature from dominating
reg_lambda	0–10	L2 regularisation on leaf values	Shrinks leaf predictions toward zero; prevents extreme predictions

The most important interaction: n_estimators and learning_rate are tightly coupled. A smaller learning rate requires more trees to achieve the same performance, but the final model almost always generalises better. The rule of thumb: set the learning rate as low as your compute budget allows, then use early stopping (below) to determine the right number of trees.

Early stopping: A technique that monitors model performance on a held-out validation set during training and stops adding trees when performance stops improving. For example, "stop if the validation error has not decreased in the last 50 rounds." This automatically determines the optimal number of trees, preventing both underfitting (too few trees) and overfitting (too many trees). It is the single most important regularisation technique for gradient boosting.

Regularisation: Any technique that prevents a model from fitting the training data too closely. Over-fitting the training data means the model memorises noise and specific quirks of the training period, rather than learning general patterns that apply to future data. Regularisation makes the model "simpler" — trading a small amount of training accuracy for much better generalisation to new data.

Feature Importance

One of the great advantages of tree-based models over neural networks is **interpretability** — we can examine which features the model considers most important for its predictions. LightGBM provides two measures of feature importance:

Split-based importance (frequency): Counts how many times each feature is used as a split variable across all trees in the ensemble. Features used frequently are deemed important. This is the simplest measure but has a significant caveat: when two features are highly correlated (e.g., demand and net load), the algorithm splits on each roughly half the time, making both appear moderately important when together they are critically important.

Gain-based importance: Measures the total reduction in the loss function attributable to splits on each feature. A feature might be used in only a few splits but produce a large improvement each time — this would show high gain importance but low split importance. Gain-based importance is generally more informative because it captures *how much* each feature helps, not just *how often* it is used.

Gradient boosting: each tree fixes the previous trees' mistakes

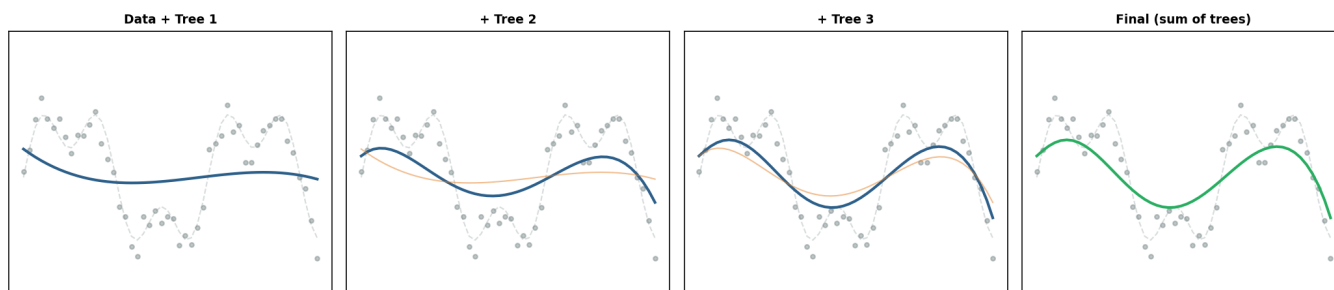


Figure 7.1 — Feature importance from a gradient-boosted tree model for NEM day-ahead price forecasting. Gain-based importance reveals which features contribute most to accurate predictions, providing a sanity check against physical intuition.

For NEM day-ahead electricity price forecasting, GBT feature importance typically reveals the following hierarchy:

1. **Lag-48 price** (yesterday's price at the same half-hour) — dominant feature
2. **Hour of day** — captures the diurnal price pattern
3. **Demand** (or net load) — the fundamental driver of the merit-order stack
4. **Lag-336 price** (same half-hour one week ago) — captures weekly seasonality
5. **Recent volatility** — signals the current price regime (calm vs. volatile)

This ordering is consistent with the stylised facts from Chapter 2, providing a valuable **sanity check**. If your model ranked an obscure feature above lag-48 price, something would likely be wrong — either a data leakage issue or an overfitting artefact.

SHAP values: A more sophisticated feature importance method based on Shapley values from cooperative game theory. Unlike global importance measures, SHAP values explain the contribution of each feature to *each individual prediction*. For example, SHAP can reveal that "for this specific spike event, the high demand feature contributed +\$200 to the prediction while the low wind feature contributed +\$150." SHAP values are additive (they sum to the difference between the prediction and the average) and theoretically grounded.

Real-world example — feature importance as a debugging tool: Suppose you train a GBT model and find that "temperature" is the most important feature — more important than lagged price. This should raise a red flag: temperature affects prices only indirectly (through demand and cooling/heating load). If temperature appears dominant, it likely means you have accidentally included a feature that contains future information (data leakage) — perhaps the actual temperature during the forecast period rather than the forecasted temperature. Feature importance is thus a diagnostic tool for detecting modelling errors.

Quantile Regression: From Point to Probabilistic

Why a Single Number Is Not Enough

Everything we have discussed so far produces a **point forecast** — a single number representing the expected (or most likely) price. But a single number conceals critical information: how confident is the model?

Point forecast: A single-valued prediction — one number per time period. For example, "the price at 6 PM tomorrow will be \$85/MWh." A point forecast gives no indication of the uncertainty around that estimate. It could mean the model is very confident (\$85, almost certainly between \$80 and \$90) or very uncertain (\$85, but it could easily be \$20 or \$500).

Quantile forecast: A set of predictions at multiple quantile levels, representing different points of the predicted distribution. For example, the 10th percentile forecast says "there is a 10% chance the price will be below this value." A set of quantile forecasts (e.g., at 5%, 10%, 25%, 50%, 75%, 90%, 95%) characterises the full shape of the model's uncertainty about the future price.

Consider two scenarios with the same point forecast of \$85/MWh:

- **Scenario A:** The model is confident. The 10th percentile is \$70, the 90th percentile is \$100. The battery should discharge — the price is reliably high no matter what happens.
- **Scenario B:** The model is uncertain. The 10th percentile is \$20, the 90th percentile is \$500. Should the battery discharge now at \$85, or wait for a possible \$500? Or should it be cautious because the price might collapse to \$20?

The point forecast is identical in both scenarios. Only the **quantile forecast** reveals the uncertainty and enables better dispatch decisions. This is why Chapter 8 (probabilistic forecasting) and Chapter 9 (forecast-to-money) are essential complements to this chapter.

Quantile Levels and Prediction Intervals

A quantile forecast produces predictions at multiple **quantile levels** (also called probability levels):

- **q = 0.05** (5th percentile): "There is a 5% chance the price will be below this value." This is the lower extreme — a very pessimistic price estimate.
- **q = 0.10**: "10% chance of being below this value."
- **q = 0.25**: "25% chance of being below — the lower quartile."
- **q = 0.50** (median): "Equally likely to be above or below this value." Not the same as the mean in skewed distributions.
- **q = 0.75**: "75% chance of being below — the upper quartile."
- **q = 0.90**: "90% chance of being below this value."
- **q = 0.95**: "95% chance of being below this value." This is the upper extreme — a very high price estimate.

Prediction interval: A range constructed from two quantile forecasts that contains the actual value with a specified probability. For example, the 80% prediction interval is $[q_{0.1}, q_{0.9}]$ — there is an 80% chance the actual price falls within this range. Wider intervals cover more outcomes but contain less information. The width of prediction intervals measures the model's **uncertainty** — narrow intervals mean the model is confident; wide intervals mean it is uncertain.

The Pinball Loss

How do you train a model to predict a specific quantile rather than the mean? The answer lies in a cleverly designed loss function called the **pinball loss** (also called the check function, tick function, or asymmetric absolute loss).

Pinball loss (check function): A loss function used to train quantile regression models. For a target quantile q , the pinball loss asymmetrically penalises underprediction and overprediction. When predicting the 90th percentile, underprediction (the actual value is above your prediction) is penalised 9 times more heavily than overprediction (the actual value is below your prediction). This asymmetry pushes the prediction upward toward the 90th percentile of the data distribution.

The pinball loss for quantile level q is:

$$L_q(y, y^*) = q \cdot \max(y - y^*, 0) + (1 - q) \cdot \max(y^* - y, 0)$$

Let us unpack this with concrete examples:

For q = 0.5 (median): The penalty for underprediction is $0.5 \times |\text{error}|$ and for overprediction is also $0.5 \times |\text{error}|$. These are equal — the loss treats under- and over-prediction symmetrically. This is just the **mean absolute error (MAE)**. Minimising it yields the median of the distribution.

For q = 0.9 (90th percentile): The penalty for underprediction ($y > y^*$) is $0.9 \times |\text{error}| = 9$ parts, while overprediction ($y^* > y$) is penalised at $0.1 \times |\text{error}| = 1$ part. Underprediction is penalised **9 times** more heavily. The model is strongly incentivised to predict high — it would rather overpredict nine times than underpredict once. This pushes the prediction toward the 90th percentile.

For q = 0.1 (10th percentile): Underprediction is penalised at 0.1 and overprediction at 0.9. The model is incentivised to predict low, yielding the 10th percentile.

Elegant mathematical property: The value of y^* that minimises the expected pinball loss at level q is *exactly* the q -th quantile of the conditional distribution of y given x . No distributional assumptions are needed — the optimisation finds the quantiles directly from the data. This makes quantile regression completely nonparametric: it works regardless of whether the price distribution is normal, skewed, heavy-tailed, or any other shape.

GBT Quantile Regression

LightGBM natively supports quantile regression by simply replacing the squared-error loss function with the pinball loss. The tree-building algorithm remains the same — the only change is what each tree is trying to predict (the negative gradient of the pinball loss rather than the residual).

To produce a full quantile forecast, you train **one separate GBT model for each quantile level**:

1. Train model₁ with $q = 0.05$ (pinball loss, $\alpha = 0.05$)
2. Train model₂ with $q = 0.10$
3. Train model₃ with $q = 0.25$
4. Train model₄ with $q = 0.50$ (median)
5. Train model₅ with $q = 0.75$
6. Train model₆ with $q = 0.90$
7. Train model₇ with $q = 0.95$

Each model uses exactly the same features — the only difference is the loss function. At prediction time, each model produces its quantile estimate, and together they form a **quantile fan** — a set of nested prediction intervals that widen as the coverage level increases.

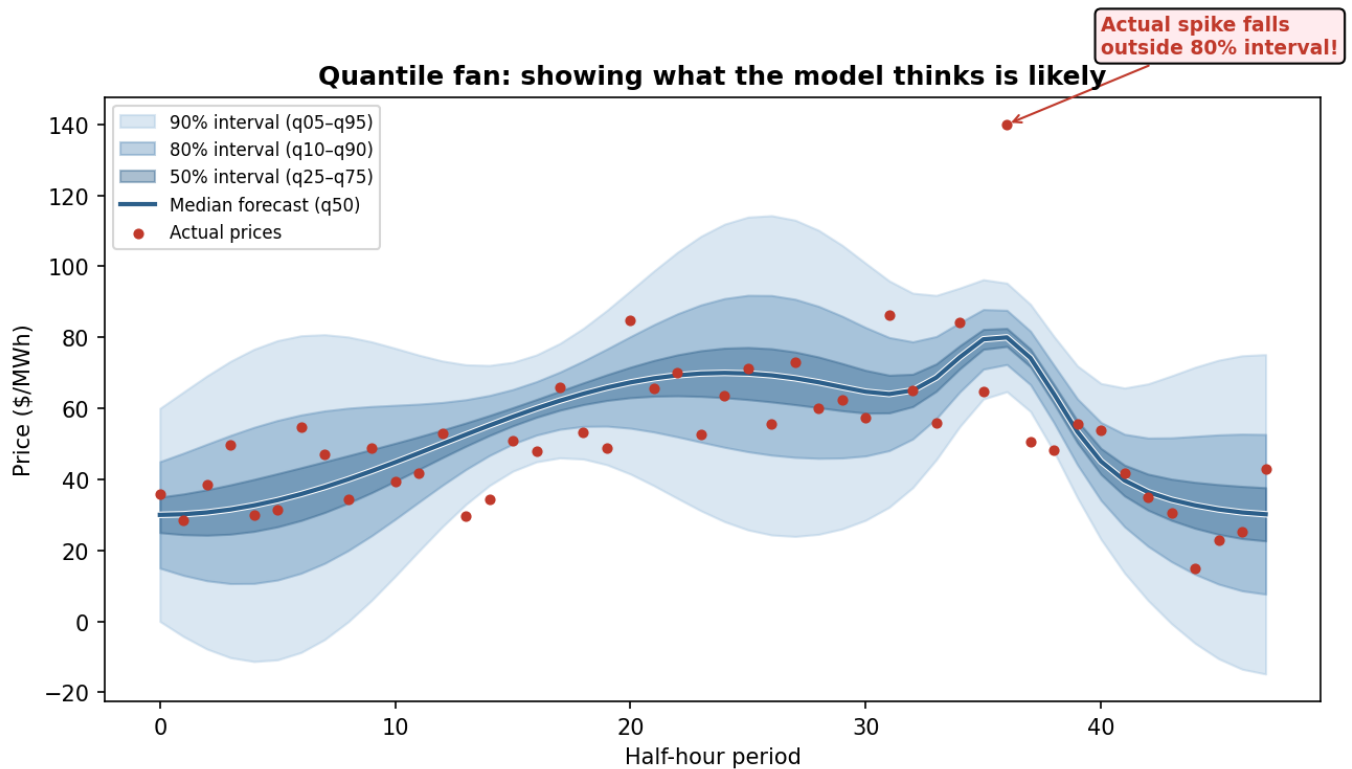


Figure 7.2 — A quantile fan produced by GBT quantile regression. The shaded bands represent prediction intervals at different coverage levels (50%, 80%, 90%, 95%). The fan widens during volatile periods (high uncertainty) and narrows during quiet periods (low uncertainty), providing a visual representation of the model's confidence over time.

Real-world example — the quantile fan in action: At 3 AM on a mild autumn night, the quantile fan is narrow: $q_{0_1_0} = \$35$, median = \$45, $q_{0_9_0} = \$55$. The model is confident — prices are predictably low during off-peak hours with mild weather. At 6 PM on a hot summer day with bushfire warnings, the fan explodes: $q_{0_1_0} = \$50$, median = \$200, $q_{0_9_0} = \$5,000$. The model knows prices will be high but cannot pin down exactly how high. A battery operator seeing this wide fan would be cautious about discharging early — there might be even higher prices coming, or the price might collapse if temperatures drop faster than expected.

The Quantile Crossing Problem

A subtle but important issue with independent quantile regression: because each quantile model is trained separately, their predictions may **cross** — the predicted 90th percentile might end up below the predicted 50th percentile for some observations. This is physically meaningless: it is impossible for the 90th percentile of a distribution to be below the median.

Quantile crossing: A situation where independently predicted quantiles violate their natural ordering — for example, the predicted 10th percentile exceeds the predicted 50th percentile. This occurs because each quantile model is optimised independently and has no constraint forcing consistency with other quantile models. While the individual quantile predictions may each be reasonable, they can be inconsistent with each other.

Quantile crossing occurs because each model is optimised independently — model₁ ($q = 0.10$) has no knowledge of model₄'s ($q = 0.50$) predictions. In practice, crossing is rare with well-trained GBT models on electricity price data, but it does occur, especially during unusual market conditions or at the extremes of the feature space.

Three common solutions:

Post-hoc sorting. After prediction, simply sort the quantiles at each time step so they are in the correct order. This is fast, simple, and effective. The downside is that sorting destroys the individual optimality of each quantile prediction — the sorted values no longer minimise their respective pinball losses.

Isotonic regression. Fit an isotonic (non-decreasing) regression across the quantile levels for each time step. This is a more principled version of sorting that finds the closest non-crossing quantile set. It is "minimally invasive" — it adjusts the quantiles as little as possible to remove crossings.

Joint quantile models. Use a neural network architecture that outputs all quantiles simultaneously, with an architectural constraint (monotonic output layer) that guarantees non-crossing by construction. This is more complex but eliminates the problem at its source.

Isotonic regression: A regression method that produces predictions constrained to be non-decreasing (or non-increasing). Given a set of data points, it finds the best-fitting monotone function — the one that minimises squared error subject to the monotonicity constraint. In the context of quantile crossing, isotonic regression ensures that quantile predictions increase with the quantile level.

Neural Networks for Price Forecasting

What Is a Neural Network?

A **neural network** is a class of models loosely inspired by the structure of biological neurons, consisting of layers of interconnected processing units. While gradient-boosted trees partition the feature space into rectangular regions, neural networks learn smooth, continuous functions through layers of simple nonlinear transformations.

Neural network: A model composed of layers of interconnected units (neurons). Each neuron computes a weighted sum of its inputs, adds a bias, and applies a nonlinear function (activation function). By stacking many neurons in multiple layers, neural networks can approximate arbitrarily complex functions. They learn by adjusting their weights to minimise a loss function, using an algorithm called **backpropagation**.

Activation function: A nonlinear function applied to the output of each neuron. Without activation functions, a multi-layer neural network would collapse to a single linear model (a stack of linear transformations is still linear). Common choices include ReLU (Rectified Linear Unit: $\max(0, x)$), sigmoid ($1/(1+e^{-x})$), and tanh. The activation function is what gives neural networks their ability to learn nonlinear relationships.

Backpropagation: The algorithm used to train neural networks. It computes how much each weight in the network contributed to the overall prediction error, then adjusts all weights simultaneously in the direction that reduces the error. The name comes from the fact that error gradients are propagated backward through the network, from the output layer to the input layer. Combined with gradient descent, backpropagation enables neural networks to learn from data.

Architecture Choices

Several neural network architectures have been applied to electricity price forecasting. Each has different strengths:

Multilayer Perceptron (MLP). The simplest architecture — fully connected layers with nonlinear activations. The input is a fixed-length feature vector (the same features as LEAR or GBT), and the output is 48 half-hourly price predictions. MLPs are suitable when features are well-engineered and the temporal structure is captured by lagged features (lag-48, lag-336) rather than by sequential processing. They are fast to train and easy to implement but have no special ability to handle sequences.

Multilayer Perceptron (MLP): A neural network consisting of an input layer, one or more hidden layers, and an output layer, where every neuron in each layer is connected to every neuron in the next layer (hence "fully connected"). It is the simplest and most general neural network architecture. Given enough hidden neurons, an MLP can theoretically approximate any continuous function — but it has no built-in understanding of sequences, spatial structure, or other data characteristics.

Recurrent Neural Networks (RNN/LSTM/GRU). These architectures process sequential data by maintaining a **hidden state** — an internal memory that carries information from earlier time steps to later ones. At each time step, the network reads the current input and updates its hidden state, allowing it to capture temporal dependencies without explicit lag features. However, RNNs are slow to train (each step depends on the previous one, preventing parallelisation), prone to gradient issues (the vanishing gradient problem makes it hard to learn long-range dependencies), and rarely outperform GBT with well-engineered features for day-ahead price forecasting.

Recurrent Neural Network (RNN): A neural network designed for sequential data, where the output at each time step depends on both the current input and the network's internal state (hidden state) from the previous time step. This creates a form of memory — the network can, in principle, use information from arbitrarily far in the past. LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) are improved variants that solve the **vanishing gradient problem** — the tendency of simple RNNs to "forget" information from more than a few time steps ago.

Vanishing gradient problem: A training difficulty in deep neural networks (and especially RNNs) where the gradients used for learning become extremely small as they are propagated backward through many layers or time steps. When gradients vanish, earlier layers or time steps learn extremely slowly or not at all — the network effectively cannot learn long-range dependencies. LSTM and GRU networks use gating mechanisms to mitigate this problem.

Convolutional Neural Networks (1D CNN). Originally designed for images, 1D CNNs apply learnable filters to sequential data. They can efficiently detect local patterns — price ramps (rapid price increases over a few intervals), spike signatures (specific patterns that precede spikes), and recurring intra-day shapes. CNNs are faster to train than RNNs because they can process all time steps in parallel. They are often used in hybrid architectures combining CNN layers (for local pattern detection) with fully connected layers (for prediction).

Convolutional Neural Network (1D CNN): A neural network that applies small, learnable filters (kernels) to segments of a sequential input, sliding the filter across the sequence to detect local patterns. Each filter learns to recognise a specific pattern (e.g., "price increasing for three consecutive intervals") wherever it appears. The same filter is applied to every position in the sequence, making CNNs parameter-efficient and good at detecting patterns regardless of where they occur.

Transformers. The most recent architecture, based on **self-attention** mechanisms that allow every time step to attend to every other time step — capturing long-range dependencies without the sequential bottleneck of RNNs. Transformers have revolutionised natural language processing (they are the "T" in GPT) but are computationally expensive for time series problems where GBT already works well. For day-ahead electricity price forecasting with well-engineered tabular features, the attention overhead is rarely justified — the problem is not complex enough to warrant the additional computational cost.

Transformer: A neural network architecture based on **self-attention** — a mechanism that allows each element in a sequence to compute a weighted combination of all other elements, with the weights learned from data. Unlike RNNs, Transformers process all positions in parallel, making them much faster to train. The self-attention mechanism can capture dependencies between any two positions regardless of distance, without the vanishing gradient problem. However, their computational cost grows quadratically with sequence length.

When Neural Networks Add Value (and When They Don't)

For day-ahead electricity price forecasting with well-engineered features, GBT typically matches or outperforms neural networks. This may seem counterintuitive — are neural networks not "more powerful"? The key insight is that **power matters only when the data is complex enough to warrant it**.

NEM day-ahead price forecasting is fundamentally a tabular problem with well-understood feature engineering. The dominant features (lagged prices, calendar variables, demand, weather) are already in a format that GBT can exploit efficiently. Neural networks add value in specific circumstances:

When neural networks help:

Raw sequential data. If you feed in raw price and demand sequences (hundreds or thousands of time steps) rather than engineered features, RNNs or Transformers can potentially learn relevant patterns. But this is rarely better than expert feature engineering — and the engineer knows things (e.g., "lag-48 is more important than lag-47") that the network must discover from scratch.

Very large datasets. Neural networks scale better to enormous datasets — their performance keeps improving with more data, while GBT's performance tends to plateau. For NEM data (typically 4–5 years $\times$ 17,520 intervals/year $\approx$ 80,000 observations), this advantage does not materialise.

Multi-output prediction. Predicting all 48 half-hours simultaneously with shared parameters can capture cross-period dependencies (e.g., if the price at 5 PM is high, the price at 6 PM is also likely high). GBT typically treats each period independently — 48 separate models with no shared learning.

Non-crossing quantiles. Neural networks can enforce non-crossing quantiles architecturally through monotonic output layers, eliminating the crossing problem without post-hoc correction.

When GBT is sufficient (most of the time):

- Feature engineering is mature (lagged prices, calendar, demand, weather)
- Training data is moderate (1–5 years)
- Interpretability matters (feature importance, SHAP values are straightforward for trees)
- Compute budget is limited (GBT trains in seconds; neural networks take minutes to hours)
- You need a reliable, low-maintenance production model

Practical guidance: Start with LightGBM. It trains in seconds, produces excellent point and quantile forecasts, provides interpretable feature importance, and requires minimal tuning beyond early stopping. Only invest in neural networks if you have a specific reason — raw sequential data, multi-task learning, or architectural constraints like non-crossing quantiles. The incremental accuracy of neural networks rarely justifies the additional complexity, training time, hyperparameter sensitivity, and maintenance burden for NEM day-ahead forecasting.

Training Neural Networks for Quantile Forecasting

Neural networks trained with the pinball loss produce quantile forecasts directly. The network output layer has one neuron per quantile per forecast period — for example, 48 periods $\times$ 9 quantiles = 432 output neurons. The loss function is the average pinball loss across all outputs and all quantile levels.

Key training considerations:

Batch normalisation. A technique that normalises the inputs to each layer during training, stabilising and accelerating learning. It helps with the scale differences between features (some in the range -5 to 10 in arcsinh space, others in the range 0 to $50,000$ in megawatts).

Batch normalisation: A technique that normalises each layer's inputs by subtracting the batch mean and dividing by the batch standard deviation. This keeps the internal values of the network on a stable scale throughout training, preventing the "internal covariate shift" problem where the distribution of inputs to each layer keeps changing as earlier layers' weights are updated.

Dropout. A regularisation technique that randomly sets a fraction of neuron outputs to zero during training. This prevents neurons from co-adapting — relying too heavily on specific other neurons — and forces the network to learn redundant representations. Typical dropout rates for price forecasting: 0.1 – 0.3 (10–30% of neurons dropped at each training step).

Dropout: A regularisation technique where, during each training step, each neuron is randomly "dropped" (its output set to zero) with a specified probability. This simulates training a different sub-network at each step, forcing the network to learn robust features that work even when some neurons are missing. At prediction time, all neurons are active but their outputs are scaled down to compensate. Dropout is the neural network equivalent of feature subsampling in gradient boosting.

Learning rate scheduling. Start with a moderate learning rate (typically 1×10^{-3}) and reduce it when the validation loss stops improving. The pinball loss landscape is piecewise linear (it has sharp corners at the predicted values), which can cause oscillation with large learning rates. Reducing the learning rate over time allows the optimiser to settle into a good solution.

Early stopping. Monitor the validation loss and stop training when it stops improving for a specified number of epochs (the "patience" parameter). This is the single most important regularisation technique for neural networks on moderate-sized datasets like NEM data. Without early stopping, the network will eventually memorise the training data — its training loss will continue to decrease while its test loss starts increasing.

Epoch: One complete pass through the entire training dataset. If the training set has 60,000 observations and the batch size is 64, one epoch consists of approximately 938 training steps (60,000 / 64). Training for 100 epochs means the network sees each training observation 100 times. Neural networks typically require tens to hundreds of epochs to converge.

Learning Curves and Model Selection

The Training Size Effect

A **learning curve** plots model performance as a function of training set size. It answers a critical practical question: "Do I have enough data for this model?"

Learning curve: A plot showing model performance (y-axis) as a function of the number of training observations (x-axis). The gap between training performance and validation performance reveals whether the model is underfitting (both lines are poor — more complexity needed) or overfitting (training is excellent but validation is poor — more data or more regularisation needed). Learning curves are essential for diagnosing model issues and determining whether collecting more data would help.

For electricity price forecasting, the learning curve reveals important differences between model families:

Small training sets (< 6 months, ~8,600 intervals): All models perform poorly because the training data does not contain a full seasonal cycle. Simple models (naive, AR) actually outperform complex ones because they have fewer parameters to estimate from limited data — a manifestation of the bias-variance tradeoff.

Medium training sets (6–18 months): LEAR catches up with and surpasses the naive. GBT begins to outperform LEAR as it has enough data to reliably learn nonlinear patterns.

Large training sets (2+ years): Performance gains flatten. Additional data helps less because the electricity market structure is slowly changing — older data becomes less relevant (generators retire, new capacity is built, demand patterns shift). This is the concept of **non-stationarity**: the statistical properties of the data change over time.

Non-stationarity: The property of a time series whose statistical characteristics (mean, variance, correlations) change over time. The NEM is non-stationary because the generation mix evolves (coal retires, renewables grow), demand patterns shift (rooftop solar changes the demand shape), and market rules change. Models trained on old data may not accurately reflect current market dynamics, which is why the most recent 1–3 years of data are typically most valuable.

The learning curve reveals the **sample efficiency** of each model — how much data each model needs to reach its potential:

Model	Sample efficiency	Explanation
Naive	N/A (doesn't learn)	Performance is flat regardless of data size
LEAR	High (few parameters)	Reaches near-optimal with ~12 months of data
GBT	Moderate (many parameters)	Continues improving up to ~2–3 years
Neural network	Low (most parameters)	Needs the most data; may not plateau within 4–5 years

Overfitting: The Central Risk

Overfitting is the most common failure mode in machine learning and deserves careful attention. An overfitting model has memorised the training data's specific quirks — including noise, outliers, and historical accidents — rather than learning the general patterns that will persist into the future.

Overfitting: A condition where a model performs excellently on training data but poorly on new, unseen data. The model has learned patterns specific to the training set that do not generalise — it has memorised rather than learned. Classic symptom: training error is much lower than test error. Overfitting is more likely with complex models (deep trees, large neural networks), small datasets, and many features.

Signs that a model is overfitting:

Training MAE is much lower than test MAE. If the model achieves MAE = \$5 on training data but MAE = \$30 on test data, it has memorised training patterns that do not generalise. A healthy gap is small — perhaps MAE = \$20 on training and MAE = \$25 on test.

Performance degrades when features are added. Adding features should help a well-regularised model. If adding weather features *hurts* test performance, the model is fitting noise in those features — the additional complexity is exploited to memorise training data more closely.

Spike predictions are too confident. The model predicts spikes that do not occur, because it has memorised specific conditions that preceded spikes in the training data. In a future period, similar conditions do not produce a spike (perhaps because a different generator is online), and the model's false spike predictions reduce its accuracy.

High variance across rolling-origin folds. If the model's MAE ranges from \$18 to \$45 across different test windows, it is highly sensitive to the specific training period — a hallmark of overfitting to particular market regimes.

Regularisation Across Model Families

Each model family has its own techniques for controlling overfitting. The following table compares the main regularisation approaches:

Technique	LEAR	GBT	Neural network
L1 penalty (drives weights to zero)	Primary tool (LASSO)	Indirect via <code>min_child_samples</code>	Rarely used
L2 penalty (keeps weights small)	Secondary (ElasticNet)	<code>reg_lambda</code> parameter	Weight decay
Feature subsampling	Not applicable	<code>colsample_bytree</code> (random feature subset per tree)	Dropout (random neuron subset per step)
Early stopping	Not needed (closed-form solution)	<code>early_stopping_rounds</code> (stop when validation loss plateaus)	patience (same concept)
Ensemble averaging	Not applicable	Built-in (the ensemble IS the model)	Can average multiple independently trained networks
Data augmentation	Not common	Subsampling (<code>subsample</code> parameter)	Adding noise to inputs

The regularisation principle: The best model is not the one that fits the training data most closely — it is the one that generalises best to unseen data. Every regularisation technique trades a small amount of training accuracy for improved test accuracy. The art of machine learning is finding the right amount of regularisation: too little and the model overfits; too much and the model underfits (it is too simple to capture the true patterns).

GBT vs Neural Network: A Practical Decision Framework

For NEM day-ahead price forecasting, the choice between GBT and neural networks is usually straightforward. Here is a decision framework:

Choose GBT (LightGBM) when:

- Your features are well-engineered tabular data (lagged prices, calendar variables, demand, weather)
- Training data spans 1–5 years (tens of thousands of observations)
- Interpretability matters for stakeholders (feature importance is trivially available)
- Compute budget is limited (GBT trains in seconds on a laptop; neural networks may need minutes to hours)
- You need a reliable, maintainable production system (GBT has fewer failure modes)
- You are a small team without deep learning expertise

Choose neural networks when:

- You are exploring end-to-end learning from raw time series (no hand-crafted features)
- Multi-task learning is beneficial (jointly predicting price, demand, and renewable generation)
- Non-crossing quantile constraints are critical for your downstream application
- You have significant compute budget and hyperparameter tuning infrastructure
- Your dataset is very large (millions of observations across many markets)
- Research novelty matters (neural network architectures offer more room for innovation)

Real-world industry context: In the electricity price forecasting community, LightGBM has become the de facto standard for practitioners and competition winners alike. The Global Energy Forecasting Competitions (GEFCom2014, GEFCom2017) were dominated by gradient-boosted tree models, often combined with linear models via QRA (Chapter 8). While academic papers frequently propose novel neural architectures, rigorous comparisons (same data, same evaluation, same computational budget) rarely show consistent advantages over well-tuned GBT. The practical conclusion: master LightGBM first, and explore neural networks only when you have a clear hypothesis about what they will add.

Glossary

Term	Definition
Linear model	A model that predicts output as a weighted sum of inputs — can only represent straight-line relationships
Nonlinear model	A model that can learn curved, threshold, or discontinuous relationships

Decision tree	A model that makes predictions via a sequence of yes/no questions (splits) on features
Leaf (terminal node)	The endpoint of a decision tree branch where a prediction is made
Split point	A feature-value threshold that divides data into two groups at a tree node
Ensemble method	A technique combining multiple weak models into a single stronger model
Gradient boosting	An ensemble method that sequentially builds trees, each correcting the errors of the previous ones
Learning rate (η)	A small multiplier controlling how much each new tree contributes to the ensemble
Residual	The difference between the actual value and the current prediction; what the next tree tries to correct
LightGBM	A fast, accurate gradient boosting implementation by Microsoft; the industry standard for tabular data
Histogram binning	Discretising continuous features into buckets to speed up split-finding in trees
Hyperparameter	A setting chosen before training (number of trees, learning rate) vs. parameters learned from data
Early stopping	Stopping training when validation performance plateaus; prevents overfitting
Regularisation	Any technique that prevents a model from fitting noise in the training data
Overfitting	When a model memorises training data specifics instead of learning generalisable patterns
Bias-variance tradeoff	The tension between model simplicity (high bias) and sensitivity to data (high variance)
Feature importance	A measure of how much each feature contributes to a tree model's predictions
SHAP values	Game-theory-based feature attribution that explains individual predictions
Point forecast	A single-valued prediction with no uncertainty information
Quantile forecast	Predictions at multiple probability levels, characterising the forecast distribution
Prediction interval	A range (e.g., 80%) expected to contain the actual value with a specified probability
Pinball loss	The asymmetric loss function used to train quantile regression models
Quantile crossing	When independently predicted quantiles violate their natural ordering
Isotonic regression	A monotone regression method; used to fix quantile crossing
MLP	Multilayer Perceptron — the simplest fully connected neural network
RNN / LSTM / GRU	Recurrent architectures that process sequential data via internal memory
Transformer	An architecture using self-attention to capture long-range dependencies in parallel
Activation function	A nonlinear function (ReLU, sigmoid) applied to neuron outputs; enables nonlinear learning
Backpropagation	The algorithm for computing gradients and updating weights in neural networks
Dropout	Regularisation that randomly deactivates neurons during training
Batch normalisation	A technique that normalises layer inputs to stabilise and accelerate training
Epoch	One complete pass through the entire training dataset
Learning curve	A plot of performance vs. training set size; reveals data sufficiency and overfitting
Non-stationarity	The property of a time series whose statistics change over time
Sample efficiency	How much data a model needs to reach its potential performance

Summary

Gradient-boosted trees (GBT) capture the nonlinear hockey-stick relationship that linear models fundamentally cannot represent. The algorithm builds an ensemble of small decision trees sequentially, each correcting the errors of its predecessors, producing a model that is both powerful and robust. LightGBM is the standard implementation, with histogram-based splitting, leaf-wise growth, and native categorical support making it fast and accurate. Key hyperparameters (learning rate, number of trees, tree depth) control the complexity of the model, and early stopping automatically finds the right balance between underfitting and overfitting. Feature importance from GBT confirms the physical intuition from earlier chapters: lagged prices and hour of day dominate, with demand and weather adding incremental value. Quantile regression via the pinball loss extends GBT from point forecasts to probabilistic forecasts — prediction intervals that quantify the model's uncertainty about future prices. Independent quantile models may produce crossing predictions, which are fixed by post-hoc sorting or isotonic regression. Neural networks (MLPs, RNNs, Transformers) are more flexible architectures but rarely outperform well-tuned GBT for day-ahead NEM price forecasting — they add value primarily for raw sequential data, multi-task learning, or architectural constraints like non-crossing quantiles. The learning curve reveals that GBT needs approximately two years of data to reach near-optimal performance, and that the marginal value of additional data diminishes as market non-stationarity makes older observations less relevant.